

Devlynix Buildathon 2.0

Official Problem Statements

Welcome to the 72-Hour MVP Sprint. Below are the official tracks and problem statements. We demand fully functional, deployed web applications. Do not build basic UI clones. Think like an executioner, build for the real world.

Global UI/UX Mandate

For all tracks, the core interface must strictly adhere to the 'Silent Coder' aesthetic: **Deep charcoal backgrounds with subtle forest green highlights**. Use clean, modern Bento-grid layouts where applicable. Avoid all neon or hyper-bright colors.

TRACK 1: EdTech Innovation | Interactive Markdown Learning Hub

The Problem: Modern technical documentation and courses need to be interactive, not just static text.

What to Build: A learning platform where 'Instructors' can upload content in MDX (Markdown with JSX) format. The platform must parse these files to render live code blocks, quizzes, and interactive React components directly within the text.

Recommended Tech Stack: Next.js (App Router), MDX, Tailwind CSS, MongoDB.

TRACK 2: Cybersecurity Tooling | Lightweight Vulnerability Scanner Dashboard

The Problem: Developers often push code without checking basic security misconfigurations.

What to Build: An automated web scanner dashboard. Users input a URL or GitHub repo link, and the backend scans it to detect outdated dependencies, missing security headers, or basic XSS injection points. The output must be a clean, visually appealing risk classification report.

Recommended Tech Stack: Node.js/Express (Backend logic), Python (optional for scanning scripts), React.js/Next.js.

TRACK 3: Developer Productivity | Intelligent Mentorship Scheduler

The Problem: Booking 1-on-1 tech mentorship sessions across different time zones without calendar conflicts is chaotic.

What to Build: A booking engine where mentors set availability and students book slots. It must include Google Calendar API integration, automatic time-zone conversion, and automated meeting link generation.

Recommended Tech Stack: MERN Stack, Google Calendar API, JWT Authentication.

TRACK 4: Gamified Education | The 'Builder & Breaker' Arena

The Problem: Practice platforms feel too academic and boring for modern web developers and security enthusiasts.

What to Build: An interactive, in-browser coding arena with two sections: 'Builder Labs' (fixing UI/API logic) and 'Breaker Labs' (Bug hunting). Must feature an embedded code editor. The layout must be highly immersive and gamified.

Recommended Tech Stack: Next.js, Monaco Editor / CodeMirror.

TRACK 5: Personal Branding | The Dynamic 'Proof-of-Work' Generator

The Problem: Resumes are dead for developers; they need dynamic portfolios, but building one from scratch takes time.

What to Build: A system where users enter their GitHub username and deployment platform tokens. The app fetches their commits, top languages, and live project links via API to auto-generate a responsive, visually stunning portfolio.

Recommended Tech Stack: React.js/Next.js, GitHub GraphQL API, Tailwind CSS.

TRACK 6: Productivity & Life-Mapping | The 'Visual Second Brain' Canvas

The Problem: College life is a chaotic mix of academics, upskilling, events, and personal goals. Traditional To-Do apps fail to show the 'Big Picture', and chat-based AI tools lack visual structure.

What to Build: An infinite visual canvas (drag-and-drop workspace) where life goals are represented as visual 'Nodes'. Features must include a 'Smart Brain-Dump' (converting unstructured text into Eisenhower Matrix nodes), visual conflict detection for overlapping deadlines, and a YouTube Action-Extractor that converts video transcripts into roadmap steps.

Recommended Tech Stack: Next.js, React Flow (crucial for visual nodes), Node.js, NLP libraries.

TRACK 7: Collaboration & Networking | Hackathon Matchmaker (Dev-Tinder)

The Problem: Finding the right team members (e.g., a backend dev looking for a UI designer) during hackathons is a major pain point.

What to Build: A real-time matchmaking platform where builders input their tech stack and GitHub profiles. Users can filter, swipe, and connect via real-time chat based on squad requirements.

Recommended Tech Stack: MERN Stack, Socket.io, Tailwind CSS.

TRACK 8: Open Source Utility | Centralized Component & Snippet Vault

The Problem: Developers waste hours rewriting the same UI components or backend configurations.

What to Build: A community-driven vault where users can upload, preview, and copy React components, Tailwind blocks, or Express setups. Must include live previews and upvote/downvote mechanics.

Recommended Tech Stack: Next.js, AWS S3/Cloudinary, MongoDB.

TRACK 9: API Infrastructure | Browser-Based API Playground

The Problem: Heavy desktop apps are overkill for simple, quick API testing during fast-paced hackathons.

What to Build: A lightweight, browser-based API testing tool. Users can send GET/POST requests, set headers, and view cleanly formatted JSON responses, all saved to local storage without requiring a login.

Recommended Tech Stack: React.js, Axios/Fetch API, Tailwind CSS.

TRACK 10: The Wildcard | Open Innovation

The Problem: Some of the best ideas don't fit into standard categories.

What to Build: Identify a massive bottleneck in the current tech ecosystem, student life, or internet security, and build a scalable MVP to solve it. Condition: Must be a deployed, functional product.

Recommended Tech Stack: Modern Component-Based Stack (MERN/Next.js highly preferred).